

Before Launching PME extension to Production

This document outlines the tasks we recommend the Protegrity team review and complete before launching the DBPS protection agent to production. These recommendations apply to both the local and server (aka remote) versions of the protection agent.

1. Review encoding for per-value encryption

The unit of work for Arrow's encryption/decryption is a Data Page. When a DBPS agent performs per-value encryption, the individual encrypted values must be "stitched" together into a single buffer to maintain the contract with Arrow. This format ultimately determines how encrypted payloads are stored on-disk.

For the sample DBPS XOR per-value encryptor, we implemented the format described in [Appendix 1](#). While this format can be reused by other implementations of the `DBPSEncryptor` interface, we recommend to review the encoding format and decide whether it is suitable for the long term needs of Protegrity's products and systems before adopting it as-is.

2. Review versioning mechanism

Upon receiving a request to encrypt a payload, DBPS returns versioning information. Specifically a `dbps_agent_version` value (currently set to `v0.01`), alongside the encrypted payload. This value is stored by Arrow as part of the encrypted data page's metadata.

When attempting to decrypt a payload, Arrow retrieves the `dbps_agent_version` from metadata and passes it to the DBPS agent as part of the `decrypt()` call. The DBPS performs a straightforward version compatibility check: the `dbps_agent_version` of the agent performing the decryption **must exactly match** the metadata version from the request. If compatible, DBPS proceeds with decryption; otherwise, an error is thrown, which, per Arrow standards, causes both the data page's and the column's decryption to fail.

Given that the versioning information is a permanent part of the stored metadata, we recommend Protegrity review and evaluate whether this versioning approach suits their long-term product strategy.

3. Authentication (DBPS server)

The DBPS server currently implements a simplified [authentication mechanism](#) using an API Client Key model. During initialization, the DBPS server loads a list of `ClientID-Key` pairs from a JSON file and these become the credentials the API client could authenticate with.

Depending on Protegrity's needs, a more sophisticated authentication mechanism may be used instead, such as using a certificate-based validation or integrating with a production Identity provider, for example.

For reference, here's a brief description of how the client passes the credentials to the DBPS server (which is the standard credential-passing flow to first obtain a session token, then using the token for subsequent calls):

To authenticate with the DBPS server, the Arrow client application provides the credentials as key-value string pairs. These credentials are forwarded unchanged to the `/token` endpoint to obtain a session JWT token, which is then used for all subsequent calls to `/encrypt` or `/decrypt`.

This client mechanism allows the credentials to be secured and read by the application as needed and passed down as-is to the DBPS server.

The flow is also described in [☰ Code-level External Encryptor + DLL interfaces/contracts](#) > Client-side/Server-side implementations.

For the local version of the agent, no DBPS server authentication is required.

4. Add support for additional compression algorithms

DBPS per-value encryption is only supported for payloads (Parquet data pages) with UNCOMPRESSED or SNAPPY compression algorithms. Parquet offers other compression algorithms (such as GZIP, ZSTD, BROTLI, among others). More encryption codecs can be integrated by extending [compression_utils](#).

When DBPS receives a payload with a non-supported compression algorithm, per-value encryption will not be attempted, and instead the payload will fallback to with per-block encryption.

5. Add support for additional Parquet encodings

Parquet uses a breadth of encodings to optimize in-memory and in-disk data representation. DBPS per-value encryption is only supported for payloads encoded with Parquet's PLAIN encoding. Parquet offers additional encodings such as DELTA_BINARY_PACKED, DELTA_LENGTH_BYTE_ARRAY, DELTA_BYTE_ARRAY, BYTE_STREAM_SPLIT.

Similar to unsupported compression algorithms, when DBPS receives a payload with a non-supported encoding, the payload will be encrypted with per-block encryption as a fallback.

6. Traversing and encrypting values

When performing per-value encryption, the XOR implementation of `DBPSEncryptor` traverses and encrypts each value sequentially (i.e. in a `for` loop). More sophisticated implementations of `DBPSEncryptor` may follow a different pattern where the individual values are encrypted in a batch fashion (in particular if the encryption requires calling an external service).

7. Production Pipeline and Setup

The DBPS server binary implements the Restful API for encryption/decryption calls and it is stateless. While it is fully functional as a standalone application server, proper infrastructure around it needs to be set up for it to be production-ready. This includes, but is not limited to, the following tasks:

- Decide on the mechanism for deploying the DBPS server code.
- Setting up network endpoints, DNS, ingress, and load balancers.
- Provisioning hardware for the estimated load to the service and horizontal scaling.
- Setting up log capture and creating dashboards for observability.
- Provisioning capacity for service dependencies (e.g. KMS, Data Protection service)

Appendices

Appendix 1: encoding for XOR-based per-value encryption

For the sample DBPS XOR encryptor we have chosen the following format to concatenate together the (per-value) individually encrypted elements.

The format is a variable format - there is a 5-byte header which applies in all scenarios, and then the encoding will vary depending on the data type of the column (to which the data belongs). Specifically, encoding for variable-sized data types (such as `BYTE_ARRAY`) will differ from that to fixed-sized data types (such as `int32`, `floats`, or `FIXED_SIZED_ARRAY`).

(1) General Header

Each payload starts with this 5-byte header

Offset	Length	Field Name	Type	Description
0x00	1 byte	<code>is_variable</code>	<code>uint8</code>	0x01 if column is variable size, 0x00 if size is fixed for all elements.
0x01	4 bytes	<code>element_count</code>	<code>uint32</code>	The total number of elements in the payload

(2) Payload Layout A: Variable size elements

If the format is variable (`is_variable == true`), each element is prefixed by its own 4-byte size header. This sequence repeats `num_elements` times.

Offset	Length	Field Name	Type	Description
0x05	4 bytes	<code>size_1</code>	uint32	Size of the first element (bytes)
0x09	<code>size_1</code>	<code>element_1</code>	bytes	Raw data of first element
...	4 bytes	<code>size_2</code>	uint32	Size of the second element (bytes)
...	<code>size_2</code>	<code>element_2</code>	bytes	Raw data of second element
...	...	...		Repeats until <code>element_N</code>

(3) Payload Layout B: Static size

If the format is static (`is_variable == false`), the size is declared only once, followed by contiguous blocks of data.

Offset	Length	Field Name	Type	Description
0x05	4 bytes	<code>element_size</code>	uint32	Size of every element in the payload
0x09	<code>element_size</code>	<code>element_1</code>	bytes	Raw data of first element
...	<code>element_size</code>	<code>element_2</code>	bytes	Raw data of second element
...	...	...		Repeats until <code>element_N</code>

Notes

- uint32 values which encode sizes follow little-endian notation.
- Payloads are naturally aligned (i.e. there is no padding).